MPOO 2027-1 · GRUPO 6 · PRÁCTICA 9 · MIÉRCOLES 28 DE OCTUBRE

La mazmorra

Encapsulamiento, primera parte. Te damos el diagrama de clases de un videojuego con caballeros, magos y arqueros; tú lo programas, lo atacas con trampas y escribes tus primeras pruebas con JUnit.

Descargar los archivos (.zip)

Qué vas a hacer

En pareja (o solo). Cinco pasos:

1. **Lee el diagrama**
   5 min · encuentra la herencia, la composición, la agregación y la asociación

2. **Prográmalo**
   25 min · siete clases, tal como dice el diagrama

3. **Ataca tu código con las trampas**
   5 min · si alguna funciona, hay un hueco

4. **Tu primera prueba con JUnit**
   10 min · una en verde y dos en rojo, a propósito

5. **Tus propias pruebas**
   5 min en clase y el resto en casa · al menos cuatro

La entrega es el **viernes 30 de octubre, 23:59**, en `p09/`.

Esta práctica sigue la próxima semana

La **P10 (miércoles 4 de noviembre)** es el complemento de esta: arranca **con tu código de La mazmorra** y le agrega herencia a las herramientas y polimorfismo. Si no terminas hoy, **termínala en casa**: sin tus siete clases funcionando, con las trampas bloqueadas y las pruebas en verde, no vas a poder hacer la P10.

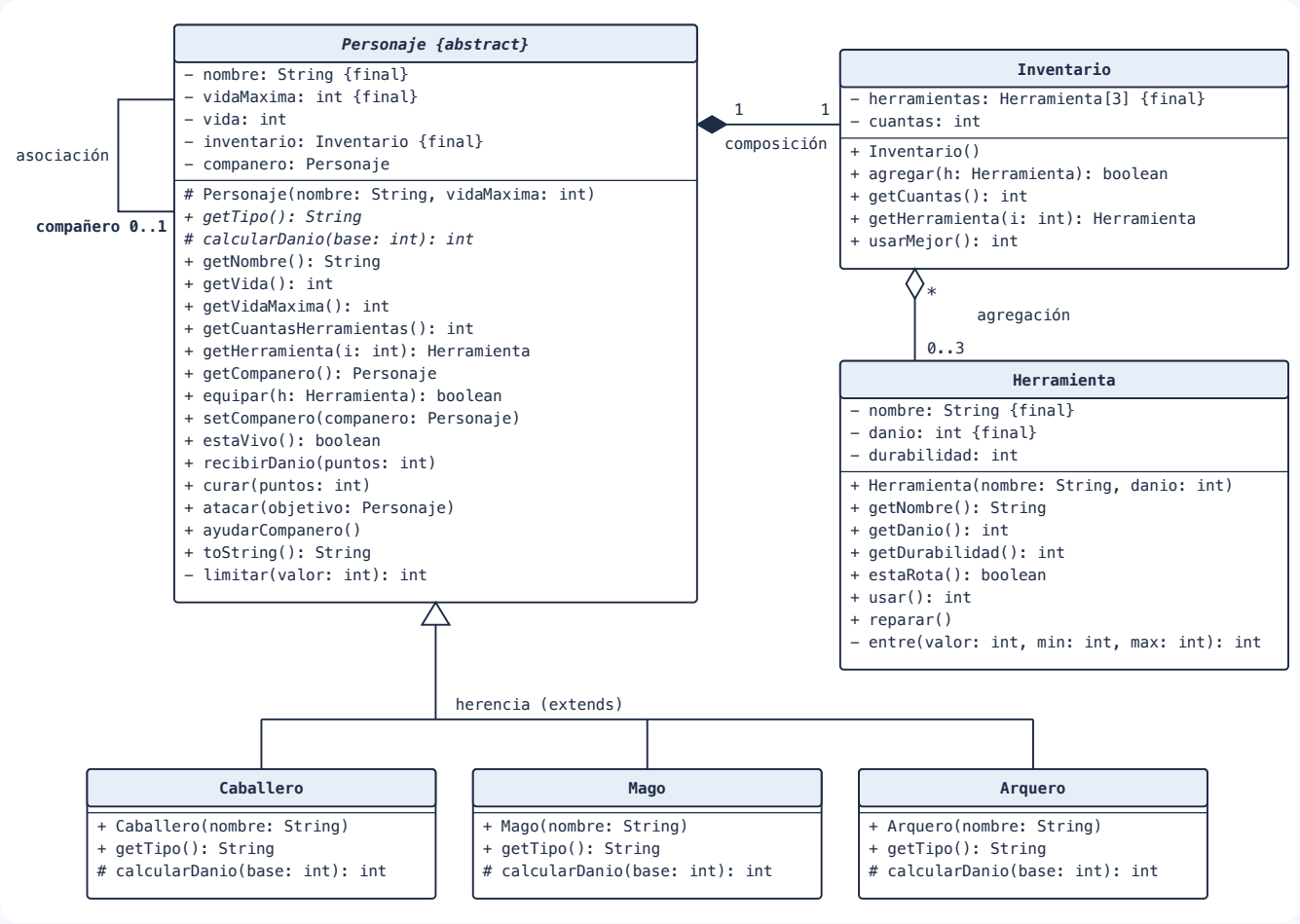
La historia

Un videojuego de mazmorras con tres tipos de personaje: el **caballero** aguanta más, el **mago** pega el doble pero aguanta menos y el **arquero** queda en medio. Cada personaje carga un **inventario** con hasta tres **herramientas** (espada, bastón, arco), que se desgastan al usarlas. Dos personajes pueden ser **compañeros** y curarse.

Hay jugadores **tramposos**: quieren vida infinita, espadas de 5000 o una cuarta herramienta. El **encapsulamiento** es lo que los detiene: los datos quedan dentro del objeto, los demás solo piden cosas por sus métodos y el objeto decide qué acepta.

Paso 1 · Lee el diagrama

Este es el diseño completo. No lo inventas tú: lo programas tal cual, como en un equipo real, donde alguien diseña y otro implementa.



Cómo se lee:

EN EL DIAGRAMA	QUIERE DECIR	EN JAVA
`-` `+` `#`	privado, público, protegido	`private`, `public`, `protected`
`{final}`	no cambia después del constructor	`final`
cursiva y `{abstract}`	clase o método abstracto	`abstract`
triángulo vacío	**herencia**: «es un»	`extends`, `super(...)`, `@Override`
◆ rombo lleno	**composición**: el dueño fabrica la parte y la parte muere con él	`new` dentro del constructor del dueño
◇ rombo vacío	**agregación**: la parte se hizo afuera y sobrevive sin el dueño	llega como parámetro y solo se guarda
línea con nombre	**asociación**: se conocen, pero ninguno es parte del otro	un atributo que se conecta después, con un `set`

`1`, `0..1`, `0..3`, `*`	cuántos objetos de cada lado	un atributo, uno que puede ser `null`, un arreglo de 3, varios

▶ **Predice: ¿quién hace el `new` del inventario? ¿Y de las herramientas?**

El diagrama dice qué hay; lo que **no** dice son las reglas. Estas son:

CLASE	REGLAS QUE TU CÓDIGO TIENE QUE CUMPLIR
`Herramienta`	El daño queda **entre 1 y 50** (lo ajusta `entre`); la durabilidad arranca en 100. `usar()`: si está rota devuelve 0; si no, gasta 1 de durabilidad y devuelve el daño. `reparar()` la deja en 100.
`Inventario`	Tiene **3 lugares**. `agregar` devuelve `false` si está lleno o si le dan `null`. `getHerramienta(i)` devuelve `null` si en esa posición no hay nada (también con -1 o 99). `usarMejor()` usa la herramienta **sana** con más daño y devuelve lo que dio; si no hay ninguna, 0. **El arreglo nunca sale de la clase.**
`Personaje`	El constructor deja la vida en su máximo y crea el inventario con `new`. La vida siempre queda **entre 0 y `vidaMaxima`** (lo ajusta `limitar`). `recibirDanio` y `curar` solo aceptan puntos mayores que 0, y **un personaje muerto no se cura**. `atacar`: si está vivo, pide `inventario.usarMejor()`; si dio más de 0, el objetivo recibe `calcularDanio(eso)`. `ayudarCompanero` cura 10 al compañero, si tiene. `toString()` devuelve `Conan (Caballero) 120/120`.
`Caballero`, `Mago`, `Arquero`	Llaman a `super(nombre, vidaMaxima)` con **120**, **80** y **100**. `getTipo()` devuelve su nombre de clase. `calcularDanio(base)`: el caballero suma 5, el mago lo duplica y el arquero suma 3.

Paso 2 · Prográmalo

Descomprime el .zip: trae `src/mazmorra/Main.java`, la carpeta `src/trampas` (para el paso 3) y la carpeta `lib` con JUnit (para el paso 4). Escribe las clases en `src/mazmorra`, cada una en su archivo y con `package mazmorra;`, en este orden:

1. `Herramienta` (no depende de nadie)
2. `Inventario` (usa `Herramienta`)
3. `Personaje` (abstracta: usa las dos)
4. `Caballero`, `Mago` y `Arquero` (cada una con `extends Personaje`)

Los nombres, tal cual el diagrama: `Main`, las trampas y las pruebas los usan.

Lo que el diagrama NO tiene, a propósito

- **No hay `setVida`**: si lo hubiera, cualquiera pondría la vida en 9999. La vida solo cambia con `recibirDanio` y `curar`, que tienen reglas.
- **No hay `getInventario` ni `getHerramientas`**: si entregaras el arreglo, cualquiera metería una cuarta herramienta sin pasar por `agregar`. Quien quiera saber algo pregunta `getCuantasHerramientas()` o `getHerramienta(i)`.
- **No todo lleva setter**: lo que es `{final}` no cambia nunca, y lo demás cambia solo por métodos con intención (`usar`, `equipar`).

Cuando tengas las siete, compila y corre `Main` desde la carpeta donde está `src`:

```
javac -d out src/mazmorra/*.java
# compila todas las clases del juego y deja los .class en la carpeta out
java -cp out mazmorra.Main
# corre el Main buscando las clases en out
```

En Windows usa `\` en vez de `/` en las rutas (`src\mazmorra\*.java`).

```
package mazmorra;

public class Main {
    public static void main(String[] args) {
        Herramienta espada = new Herramienta("Espada", 20);
        Herramienta baston = new Herramienta("Baston", 10);
        Herramienta arco = new Herramienta("Arco", 15);

        Personaje conan = new Caballero("Conan");
        Personaje merlin = new Mago("Merlin");
        Personaje legolas = new Arquero("Legolas");
        conan.equipar(espada);
        merlin.equipar(baston);
        legolas.equipar(arco);
        merlin.setCompanero(conan);

        System.out.println("Al empezar:");
        System.out.println(conan);
        System.out.println(merlin);
        System.out.println(legolas);

        conan.atacar(legolas);
        merlin.atacar(conan);
        legolas.atacar(merlin);
        merlin.ayudarCompanero();

        System.out.println("Despues de una ronda:");
        System.out.println(conan);
        System.out.println(merlin);
        System.out.println(legolas);
        System.out.println(espada.getNombre() + ": durabilidad " +
espada.getDurabilidad());
    }
}
```

▶ **Predice: antes de correrlo: ¿cuánta vida le queda a Legolas después de la ronda?**

Merlín pega **20** con un bastón de 10 porque el mago duplica; Conan queda en 100 y sube a **110** porque Merlín es su compañero y lo ayuda; Legolas pega 15 + 3 = **18**. La espada bajó a **99**.

Paso 3 · Ataca tu código con las trampas

En `src/trampas` hay dos tramposos. Viven en **otro paquete**, como cualquier jugador de afuera.

3a · Trampas de acceso: este archivo NO debe compilar

```
package trampas;

import mazmorra.*;

// Trampas de acceso: este archivo NO debe compilar.
// Cada error del compilador es una trampa bloqueada.
public class TrampasDeAcceso {
    public static void main(String[] args) {
        Personaje heroe = new Caballero("Heroe");
        heroe.vida = 9999;
        heroe.inventario = new Inventario();

        Herramienta espada = new Herramienta("Espada", 20);
        espada.durabilidad = 1000000;

        Inventario bolsa = new Inventario();
        bolsa.herramientas[0] = espada;
        bolsa.cuantas = 50;
    }
}
```

```
javac -cp out -d out src/trampas/TrampasDeAcceso.java
# intenta compilar el tramposo usando tus clases, que ya están en out
```

> **DEBE SALIR**
>
> ```
> TrampasDeAcceso.java:10: error: vida has private access in Personaje
> TrampasDeAcceso.java:11: error: inventario has private access in Personaje
> TrampasDeAcceso.java:14: error: durabilidad has private access in Herramienta
> TrampasDeAcceso.java:17: error: herramientas has private access in Inventario
> TrampasDeAcceso.java:18: error: cuantas has private access in Inventario
> ```

Cada error es una trampa bloqueada: «ese atributo es privado, no lo puedes tocar». Si alguna de esas líneas **sí compila**, ese atributo no es `private` en tu código: corrígelo. Que este archivo no compile es lo esperado; no lo arregles.

3b · Trampas de reglas: estas sí compilan, y tu código tiene que ignorarlas

```
javac -cp out -d out src/trampas/Tramposo.java
# compila el segundo tramposo
java -cp out trampas.Tramposo
# lo corre: intenta siete trampas y dice cuáles funcionaron
```

> **DEBE SALIR**
>
> ```
> BLOQUEADA  1) golpe negativo para curarse
> BLOQUEADA  2) curarse arriba del tope de vida
> BLOQUEADA  3) forjar una espada de 5000 de poder
> BLOQUEADA  4) cargar una cuarta herramienta
> BLOQUEADA  5) pedir la herramienta 99
> BLOQUEADA  6) atacar sin ninguna herramienta
> BLOQUEADA  7) revivir a un personaje muerto
> ```

Si alguna dice **FUNCIONA**, encontraste un hueco:

TRAMPA	SI DICE FUNCIONA, REVISA
1 · golpe negativo para curarse	`recibirDanio`: solo puntos mayores que 0
2 · curarse arriba del tope	`curar` y `limitar`: el tope es `vidaMaxima`, no 100
3 · espada de 5000	el constructor de `Herramienta` y `entre`

4 · cuarta herramienta	`agregar`: tres lugares y luego `false`
5 · herramienta 99	`getHerramienta`: posición fuera del rango, `null`
6 · atacar sin herramienta	`usarMejor` devuelve 0 y `atacar` no hace daño con 0
7 · revivir a un muerto	`curar`: un personaje muerto no se cura

Si en lugar de terminar truena con `ArrayIndexOutOfBoundsException`, la trampa 5 encontró el hueco: tu `getHerramienta` buscó en una posición que no existe. Cada vez que cambies algo, vuelve a compilar el juego (paso 2) y el tramposo.

Paso 4 · Tu primera prueba con JUnit

Una **prueba** es un método que arma un objeto, le hace algo y **afirma** qué debería pasar. Si lo afirmado es cierto, sale en **verde**; si no, en **rojo**. El tramposo lo tuviste que leer tú; una prueba la revisa la computadora, cada vez que quieras.

1. **Crea la carpeta de pruebas**
   Junto a `src` crea `test`, y dentro `mazmorra`: `test/mazmorra/`.

2. **Crea `PersonajeTest.java`**
   Dentro de `test/mazmorra`, con el código de abajo.

3. **Compila y corre**
   Con los dos comandos de abajo. JUnit es el `.jar` de la carpeta `lib`.

```
package mazmorra;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class PersonajeTest {
    @Test
    void laVidaNuncaPasaDeLaMaxima() {
        Personaje p = new Caballero("Conan");
        p.curar(50);
        assertEquals(120, p.getVida());
    }
}
```

`@Test` marca el método como prueba. `assertEquals(120, p.getVida())` afirma que lo **esperado** (primero) es igual a lo **real** (segundo).

```
javac -cp lib/junit-platform-console-standalone-1.11.4.jar -d out
src/mazmorra/*.java test/mazmorra/*.java
# compila el juego y las pruebas; -cp le dice dónde está JUnit
java -jar lib/junit-platform-console-standalone-1.11.4.jar execute --
class-path out --scan-class-path
# corre todas las pruebas que encuentre en out
```

▶ **Predice: ¿en qué color sale?**

4b · Tu primera prueba en rojo

Abajo de la primera, dentro de la misma clase, agrega esta. Compila y corre otra vez:

```
    @Test
    void recibirDanioRestaLosPuntos() {
        Personaje p = new Mago("Merlin");
        p.recibirDanio(30);
        assertEquals(70, p.getVida());
    }
```

VERÁS

```
╷
├─ JUnit Jupiter ✔
│  └─ PersonajeTest ✔
│     ├─ laVidaNuncaPasaDeLaMaxima() ✔
│     └─ recibirDanioRestaLosPuntos() ✘ expected: <70> but was: <50>
    => org.opentest4j.AssertionFailedError: expected: <70> but was: <50>
       mazmorra.PersonajeTest.recibirDanioRestaLosPuntos(PersonajeTest.java:18)
[         1 tests successful      ]
[         1 tests failed          ]
```

Dice «esperaba 70, pero salió 50». **¿Quién está mal: el código o la prueba?**

▶ **Predice: ¿quién está mal?**

4c · Rompe el código a propósito

Ahora una prueba buena que descubre un código malo. En `Personaje`, deja `limitar` así (comenta el original, no lo borres):

```
    private int limitar(int valor) {
        return valor;   // el Math.max y Math.min ya no están
    }
```

Compila y corre: la primera prueba sale en rojo con `expected: <120> but was: <170>`. Esta vez el error **sí** estaba en el código, y la prueba lo atrapó sin que corrieras nada más. **Regresa** `limitar` y comprueba que vuelve a verde.

Paso 5 · Tus propias pruebas

Escribe **al menos cuatro pruebas propias** en `PersonajeTest` o en clases nuevas (`InventarioTest`, `HerramientaTest`) dentro de `test/mazmorra`. Cada una arma objetos, les hace algo y afirma. Un nombre que diga lo que comprueba: `unMuertoNoSeCura`, no `prueba2`. Al menos **una de herencia** y **una de una relación**. Ideas:

QUÉ COMPRUEBA	IDEA
Una trampa	convierte una trampa del tramposo en prueba: una espada de 5000 tiene daño 50; la cuarta herramienta devuelve `false`; `getHerramienta(99)` da `null`
Herencia	cada tipo empieza con su vida (120, 80, 100); un caballero con espada de 20 le quita 25 a un arquero; un mago con bastón de 10 le quita 20
Composición	cada personaje tiene **su propio** inventario: uno equipa dos herramientas y el otro sigue con 0
Agregación	dos personajes equipan **la misma** hacha: `assertSame` sobre las dos, y cuando uno ataca, la durabilidad de la hacha del otro también baja
Asociación	un mago con un compañero herido: al ayudarlo, el compañero sube 10

AFIRMACIÓN	CUÁNDO
`assertEquals(esperado, real)`	los dos valores deben ser iguales
`assertTrue(...)` y `assertFalse(...)`	un `boolean`, como `estaVivo()` o lo que devuelve `equipar`
`assertNull(...)`	debe ser `null`, como `getHerramienta(99)`
`assertSame(a, b)`	`a` y `b` son **el mismo objeto**, no dos iguales

Si algo truena

No compila

- `Caballero is not abstract and does not override abstract method calcularDanio(int) in Personaje`: a una hija le falta uno de los dos métodos abstractos (`getTipo` o `calcularDanio`).
- `constructor Personaje in class Personaje cannot be applied to given types`: a la hija le falta `super(nombre, 120)` como primera línea del constructor.
- `attempting to assign weaker access privileges; was protected`: escribiste `calcularDanio` como `private` en la hija; tiene que ser `protected` (o `public`), igual o más abierto que en el padre.
- `package org.junit.jupiter.api does not exist`: al compilar las pruebas te faltó `-cp lib/junit-platform-console-standalone-1.11.4.jar` .
- `cannot find symbol` sobre `Caballero` en la prueba: la prueba debe estar en `test/mazmorra` y empezar con `package mazmorra;` .

Compila, pero algo sale distinto

- **Conan termina en 100 y no en 110**: `ayudarCompanero` no cura, o en `Main` falta `setCompanero` .
- **Merlín pega 10 en vez de 20**: `atacar` usa el daño de la herramienta directo; tiene que pasar por `calcularDanio` .
- **Una prueba roja**: lee `expected ... but was ...` antes de tocar nada, y revisa los dos lados: el código y lo que la prueba afirma.

La entrega

QUÉ	DETALLE
Cuándo	**Viernes 30 de octubre, 23:59.** Sin IA, como siempre.

Con quién	En pareja (o solo). **Cada integrante sube todo en su propia carpeta.**
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p09/`. Solo push.
Qué	Tus carpetas `src` y `test`, y una **bitácora en PDF**

La bitácora lleva: los integrantes (nombre y número de cuenta); capturas de la salida de `Main`, del `Tramposo` con las siete **BLOQUEADA**, de los errores de `TrampasDeAcceso` y de **todas tus pruebas en verde**; y estas tres respuestas con tus palabras:

1. **Las cuatro relaciones:** para herencia, composición, agregación y asociación, copia la línea de tu código donde se ve cada una y di qué le pasa a la parte si el dueño desaparece (para la herencia, qué hace `super(...)`).
2. **Los huecos:** ¿alguna trampa funcionó la primera vez? ¿Qué cambiaste? Si ninguna funcionó, elige una y explica qué línea de tu código la bloquea. ¿Por qué el diagrama no tiene `setVida` ni `getHerramientas`?
3. **Una prueba roja:** copia el mensaje del paso 4b y explica qué significa cada número y quién estaba mal.

Cómo se califica, sobre 10:

QUÉ REVISO	PUNTOS
Fiel al diagrama: las siete clases con sus visibilidades, `final` y `abstract`; las hijas con `super(...)` y `@Override`; el inventario con `new` adentro; las herramientas por `equipar`; el compañero por `setCompanero`	3
Encapsulamiento: `TrampasDeAcceso` no compila por los cinco atributos y el `Tramposo` da siete BLOQUEADA	2.5
Corre y la salida de `Main` es la de la guía	1
Pruebas: la de la guía y al menos cuatro propias (una de herencia y una de relación), todas en verde y con nombres que dicen qué comprueban	2
Bitácora: las capturas y las tres respuestas	1.5

Cada comando de abajo, con lo que hace:

```
git checkout entregas_apellido_nombre
# te pasa a tu rama de entregas
git pull origin main
# trae lo nuevo del repositorio del grupo
mkdir -p entregas/apellido_nombre/p09
# crea tu carpeta de esta práctica
# copia aquí src, test y la bitácora en PDF
git add entregas/apellido_nombre/p09
# marca lo que vas a entregar
git commit -m "P9: encapsulamiento, primera parte"
# lo guarda con un mensaje
git push
# lo sube: esto es la entrega
```

Guarda tu proyecto completo (con `lib` y `src/trampas`): la P10 empieza justo donde termina esta.

Si los `.class` de `out` se cuelan, no pasa nada; no hace falta subirlos. El código tiene que ser suyo; si algo no sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM

Prof. Oscar Manuel Ruiz Hurtado · Práctica 9 · Encapsulamiento, primera parte (MADO-21, guía 9: datos y métodos miembro, modificadores de acceso, métodos de acceso, composición) con herencia, agregación, asociación e introducción a JUnit · octubre de 2026